# Control de un robot mediante gestos de la mano con una red convolucional entrenada desde cero

Daniel Rodríguez Alonso  Carlos Falcón Brito

Aprendizaje Automático 2 · Práctica de redes convolucionales
31 de octubre de 2026

**Resumen**

Este trabajo estudia si una red convolucional pequeña, entrenada desde cero, puede reconocer con fiabilidad un vocabulario de gestos de la mano pensado para dar órdenes a un robot. Partimos de un subconjunto de HaGRID con ocho gestos (*stop*, *like*, *dislike*, *one*, *peace*, *fist*, *palm* y *call*) y una clase nula (*no_gesture*) que deja el robot quieto. Clasificamos recortes de la mano de $128 \times 128$ px y dividimos los datos *por persona*, de modo que las 726 personas de test no aparecen en entrenamiento ni en validación. La red propuesta, GestureCNN, tiene cinco bloques convolucionales con normalización por lotes, *dropout* y *global average pooling* (2,36 M parámetros). Tras una búsqueda manual de 9 configuraciones, el modelo final se entrena con lotes de 32. En test alcanza una exactitud del 99,03 % (IC del 95 % por *bootstrap* por persona: [98,61 %; 99,42 %]) y una F1 macro del 99,03 %, muy por encima del objetivo del 85 %. Un estudio de ablación con dos semillas indica que la normalización por lotes es el componente decisivo, mientras que el *dropout* y el aumento de datos tienen efectos pequeños sobre la exactitud con este conjunto de datos; la búsqueda de hiperparámetros muestra también una sensibilidad baja a la tasa de aprendizaje, al lote, a la capacidad y a la activación. Un ResNet18 preentrenado en ImageNet sirve de referencia (99,20 % en test) con 4,7 veces más parámetros. Los mapas Grad-CAM muestran que la red atiende a los dedos que distinguen cada gesto, y los errores restantes se concentran en pares visualmente próximos (*stop*/*palm*, *like*/*call*) y en ejemplos con ruido de etiqueta. Por último, se describe el diseño de una demo en tiempo real con ROS 2 y *turtlesim*.

**Palabras clave:** reconocimiento de gestos, redes convolucionales, interacción persona-robot, HaGRID, ablación, Grad-CAM.

## 1. Introducción

Los gestos de la mano son una forma natural de dar órdenes a una máquina sin mando ni contacto. Esta ventaja es especialmente útil en entornos ruidosos o industriales, donde la voz no es fiable, o cuando el operador tiene las manos ocupadas y no puede manejar un mando [1]. Para un robot, sin embargo, no basta con reconocer gestos: hace falta un *vocabulario de órdenes* con un significado preciso para cada uno y, sobre todo, una forma segura de no hacer nada. Por eso este trabajo plantea el problema como la clasificación de un conjunto cerrado de órdenes más una *clase nula* (*no_gesture*) que deja el robot quieto, en lugar de como un reconocimiento de gestos arbitrarios.

Este planteamiento impone tres requisitos que guían todo el diseño experimental:

1. **Robustez ante personas nuevas.** El robot tendrá que obedecer a personas que el modelo nunca vio. Por tanto, la estimación del rendimiento debe hacerse con personas ausentes del entrenamiento; una partición aleatoria por imagen sería optimista, porque la misma persona, con el mismo fondo y la misma iluminación, aparecería a ambos lados [2].

2. **Una clase nula fiable.** Confundir una mano en reposo con una orden mueve el robot sin querer, que es el error más costoso en la práctica.
3. **Bajo coste de inferencia.** La clasificación debe ejecutarse en tiempo real en un ordenador portátil, lo que favorece modelos pequeños.

**Objetivos.** El objetivo principal es alcanzar al menos un 85 % de exactitud en test con personas no vistas, usando una red convolucional diseñada y entrenada desde cero en PyTorch [3]. La red debe incluir convolución, *pooling*, normalización por lotes y *dropout*, y su entrenamiento debe usar parada anticipada sobre la pérdida de validación con restauración de los mejores pesos. Además, nos propusimos analizar el modelo con una matriz de confusión, mapas Grad-CAM y un estudio de ablación (con y sin *dropout*, normalización por lotes y aumento de datos), y documentar una búsqueda manual de hiperparámetros. Como extensiones opcionales, el plan incluía comparar con un ResNet18 preentrenado y construir una demo en tiempo real que controle un robot simulado en ROS 2.

**Contribuciones.**

- Un *pipeline* de datos reproducible sobre HaGRID: recorte de manos a partir de las cajas anotadas, una clase nula construida con las manos que no hacen el gesto y una partición estratificada *por persona* sin personas compartidas entre particiones (apartado 3).
- GestureCNN, una red de 2,36 M parámetros que alcanza un 99,03 % de exactitud en test, con un intervalo de confianza que tiene en cuenta la dependencia entre imágenes de una misma persona (apartado 5).
- Una búsqueda manual de hiperparámetros (apartado 5.1) y un estudio de ablación con dos semillas y pruebas de McNemar pareadas, que separa los efectos reales de la variación entre ejecuciones (apartado 5.4).
- Una comparación con ResNet18 que incluye el coste computacional (parámetros, operaciones y latencia) y un análisis cualitativo de los errores con Grad-CAM.

El resto del documento se organiza así: la apartado 2 resume el trabajo relacionado; la apartado 3 describe los datos y la partición; la apartado 4, el preprocesado, la arquitectura y el protocolo experimental; la apartado 5 presenta los resultados; la apartado 6, el diseño de la demo; la apartado 7 discute los resultados y sus limitaciones, y la apartado 8 concluye.

## 2. Trabajo relacionado

**Conjuntos de datos de gestos.** HaGRID [4] es uno de los mayores conjuntos de imágenes de gestos de la mano: su primera versión contiene 552 992 imágenes FullHD de 18 gestos pensados para controlar dispositivos, tomadas por miles de personas en entornos domésticos y con condiciones de iluminación muy variadas. Cada imagen incluye las cajas de las manos, la etiqueta de cada mano (las manos que no hacen el gesto se etiquetan como *no_gesture*) y un identificador anónimo de la persona (`user_id`), lo que permite particionar por persona. Usamos un subconjunto público de 30 000 imágenes reescaladas a 384 px [5].

**Detector seguido de clasificador.** En una imagen completa la mano ocupa muy pocos píxeles, así que los sistemas prácticos suelen dividir el problema en dos etapas: un detector o rastreador de manos, como MediaPipe Hands [6], localiza la mano, y un clasificador decide el gesto a partir del recorte [1]. Este trabajo se centra en la segunda etapa: durante el entrenamiento y la evaluación usamos las cajas anotadas, y el detector solo interviene en la demo (apartado 6).

**Arquitecturas convolucionales.** Nuestra red sigue el esquema de bloques de VGG [7]: dos convoluciones $3 \times 3$ seguidas de *max pooling*. Le añade normalización por lotes (BN) [8], que estabiliza la distribución de las activaciones y permite tasas de aprendizaje mayores, y *dropout* [9], en su variante espacial (`Dropout2d`) en los bloques convolucionales [10]. En lugar de capas densas grandes al final usa *global average pooling* [11], que reduce mucho el número de parámetros. Como referencia de un modelo más grande usamos ResNet18 [12] preentrenado en ImageNet [13]; ajustar una red preentrenada aprovecha rasgos genéricos de las primeras capas que se transfieren bien a otras tareas [14].

**Explicabilidad y evaluación.** Grad-CAM [15] genera mapas de calor que indican qué regiones de la imagen empujan la predicción hacia una clase, a partir de los gradientes en la última capa convolucional. Lo usamos para comprobar que la red mira la mano y no el fondo. Para comparar clasificadores evaluados sobre el mismo conjunto de test usamos la prueba de McNemar [16], recomendada para este escenario por Dietterich [17], e intervalos de confianza por *bootstrap* [18].

# 3. Datos

## 3.1. Subconjunto de HaGRID y vocabulario de órdenes

Partimos del subconjunto `hagrid-sample-30k-384p` [5], que conserva las anotaciones originales (cajas en coordenadas normalizadas $[x, y, w, h]$ con origen en la esquina superior izquierda, etiqueta por mano y `user_id`) con imágenes reescaladas a 384 px en su lado mayor. El plan inicial preveía una versión a 512 px; elegimos la de 384 px porque es la que está publicada con las cajas y el identificador de persona y su descarga es mucho menor. El coste de esta elección es que las manos son pequeñas: el lado mayor de la caja tiene una mediana de 68 px (el 10 % de las manos mide menos de 36 px), de modo que la mayoría de los recortes se *amplían* hasta $128 \times 128$ px y pierden detalle fino. Volveremos a este punto en la apartado 7.

De los 18 gestos del subconjunto seleccionamos los ocho que forman el vocabulario de órdenes de la tabla 1. Los elegimos porque cubren las órdenes básicas de un robot móvil con pinza y porque son, en su mayoría, visualmente distintos entre sí; la excepción conocida es el par *stop*/*palm*, que solo se diferencia en si los dedos están juntos o separados. Las 14.078 imágenes de esos ocho gestos contienen 17.296 manos anotadas de 4.877 personas. La clase nula *no_gesture* no procede de una carpeta propia: son las 3.219 *segundas manos* de esas mismas imágenes, que HaGRID anota como *no_gesture* porque no hacen el gesto (cajas rojas en la figura 1). Esta construcción tiene una ventaja importante: los ejemplos negativos comparten persona, fondo e iluminación con los positivos, así que la red no puede distinguir las clases por el contexto y debe fijarse en la forma de la mano.

## 3.2. Equilibrado y partición por persona

Con 3.219 ejemplos, la clase nula casi duplica a cualquier clase de gesto. Para no sesgar la red hacia ella, la submuestreamos aleatoriamente (semilla fija) hasta el tamaño medio de una clase de gesto, lo que deja 1.760 ejemplos. El conjunto final tiene 15.837 recortes de 4.877 personas.

La partición se hace con `StratifiedGroupKFold` de scikit-learn en 20 trozos de aproximadamente el 5 % de los datos, usando `user_id` como grupo: cada persona cae entera en un único trozo y cada trozo mantiene la proporción de clases. Asignamos 14 trozos a entrenamiento, 3 a validación y 3 a test, es decir, una partición 70/15/15. El resultado (tabla 1) tiene 11.088 recortes de entrenamiento (3.416 personas), 2.375 de validación (735 personas) y 2.374 de test (726 personas), con 0 personas compartidas entre particiones; una prueba automática (`tests/test_splits.py`)

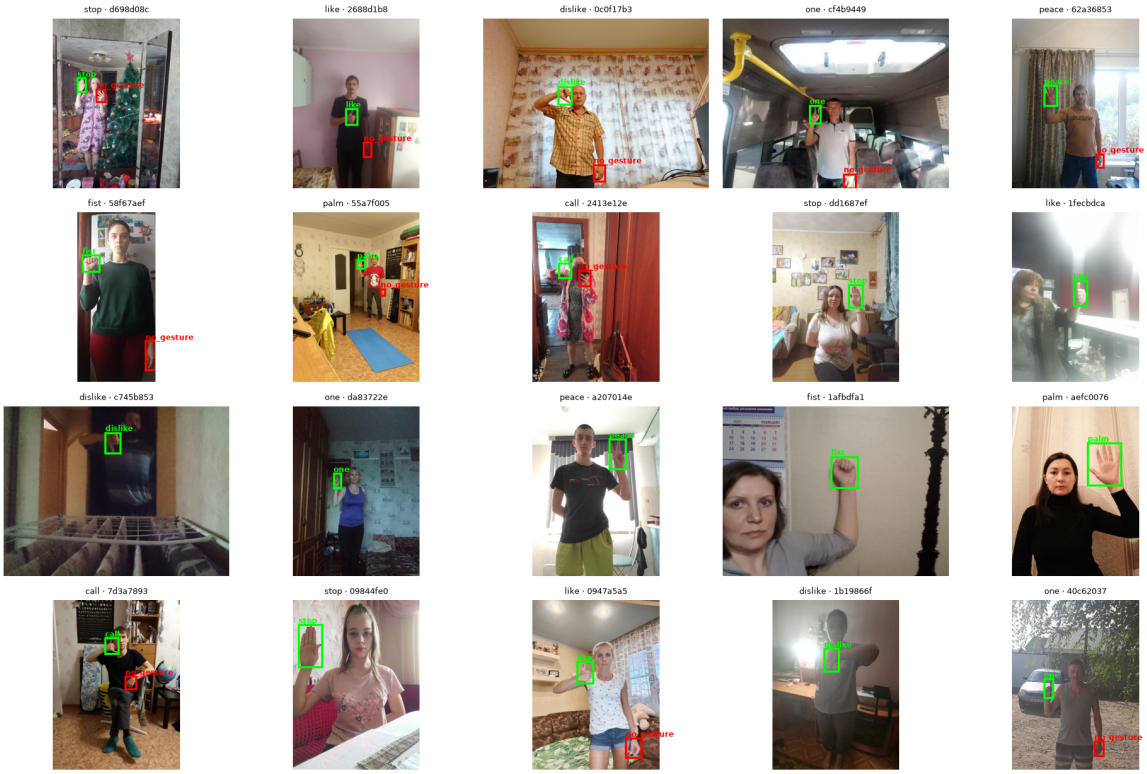



**Figura 1:** Imágenes de HaGRID con sus anotaciones. En verde, la mano que hace el gesto; en rojo, la segunda mano, etiquetada como *no_gesture*. La mano ocupa una fracción muy pequeña de la imagen, lo que justifica clasificar recortes y no el fotograma completo.

lo comprueba. Las clases quedan equilibradas en las tres particiones, por lo que la exactitud y la F1 macro son métricas comparables y no hace falta ponderar la función de pérdida.

Por último, la media y la desviación típica por canal que se usan para normalizar las imágenes se calculan *solo* con la partición de entrenamiento, para que ninguna información de validación o test intervenga en el entrenamiento.

**Tabla 1:** Vocabulario de órdenes y número de recortes por clase y partición, tras equilibrar *no_gesture*. La partición es por persona: ninguna persona aparece en más de una columna.

| Clase | Orden | Train | Val | Test | Total |
|---|---|---|---|---|---|
| *stop* | Parar | 1225 | 261 | 262 | 1748 |
| *like* | Avanzar | 1214 | 258 | 260 | 1732 |
| *dislike* | Retroceder | 1248 | 268 | 267 | 1783 |
| *one* | Girar a la izquierda | 1245 | 267 | 266 | 1778 |
| *peace* | Girar a la derecha | 1237 | 266 | 266 | 1769 |
| *fist* | Cerrar pinza | 1214 | 259 | 261 | 1734 |
| *palm* | Abrir pinza | 1239 | 267 | 264 | 1770 |
| *call* | Volver al inicio | 1234 | 265 | 264 | 1763 |
| *no_gesture* | Ninguna (quieto) | 1232 | 264 | 264 | 1760 |
| Total imágenes | | 11088 | 2375 | 2374 | 15837 |
| Personas | | 3416 | 735 | 726 | 4877 |

# 4. Métodos

## 4.1. Preprocesado

Cada mano se recorta como un cuadrado centrado en su caja, con lado igual al lado mayor de la caja más un margen del 25 % por cada lado (lado = máx$(w, h) \cdot 1{,}5$). El recorte cuadrado evita deformar la mano al redimensionarla, y el margen conserva el contexto inmediato (muñeca, dedos que sobresalen de la caja) y deja espacio para el aumento de datos. Las zonas que caen fuera de la imagen se rellenan con negro. Los recortes se redimensionan a $128 \times 128$ px, se guardan en JPEG de alta calidad y, al cargarlos, se normalizan por canal con las estadísticas de entrenamiento. La figura 2 muestra ejemplos de todas las clases.

## 4.2. Aumento de datos

Durante el entrenamiento, cada recorte pasa por cuatro transformaciones aleatorias [19], elegidas para no alterar el significado del gesto (figura 3):

- **Rotación** uniforme de hasta $\pm 15°$: la mano nunca está perfectamente vertical. No se rota más porque una rotación grande podría convertir *like* en algo parecido a *dislike* o a un pulgar horizontal.
- **Recorte aleatorio con reescalado** (`RandomResizedCrop`) que conserva entre el 80 % y el 100 % del área y la relación de aspecto 1:1. Simula pequeños errores de centrado del detector; como el margen es del 25 %, la mano no llega a cortarse.
- **Volteo horizontal** con probabilidad 0,5, que intercambia mano derecha e izquierda sin cambiar el gesto. El volteo *vertical* se excluye porque convierte *like* en *dislike*.
- **Variación de color**: brillo y contraste $\pm 30$ %, saturación $\pm 20$ %. No se altera el tono para no generar colores de piel irreales.

Validación y test solo se normalizan, sin transformaciones aleatorias.

## 4.3. Arquitectura: GestureCNN

GestureCNN (figura 4) encadena cinco bloques convolucionales. Cada bloque aplica dos veces la secuencia convolución $3 \times 3$ (con *padding* 1) $\rightarrow$ BN $\rightarrow$ ReLU, y termina con un *max pooling* $2 \times 2$ que divide la resolución por dos. Los canales crecen como 32, 64, 128, 256 y 256, de modo

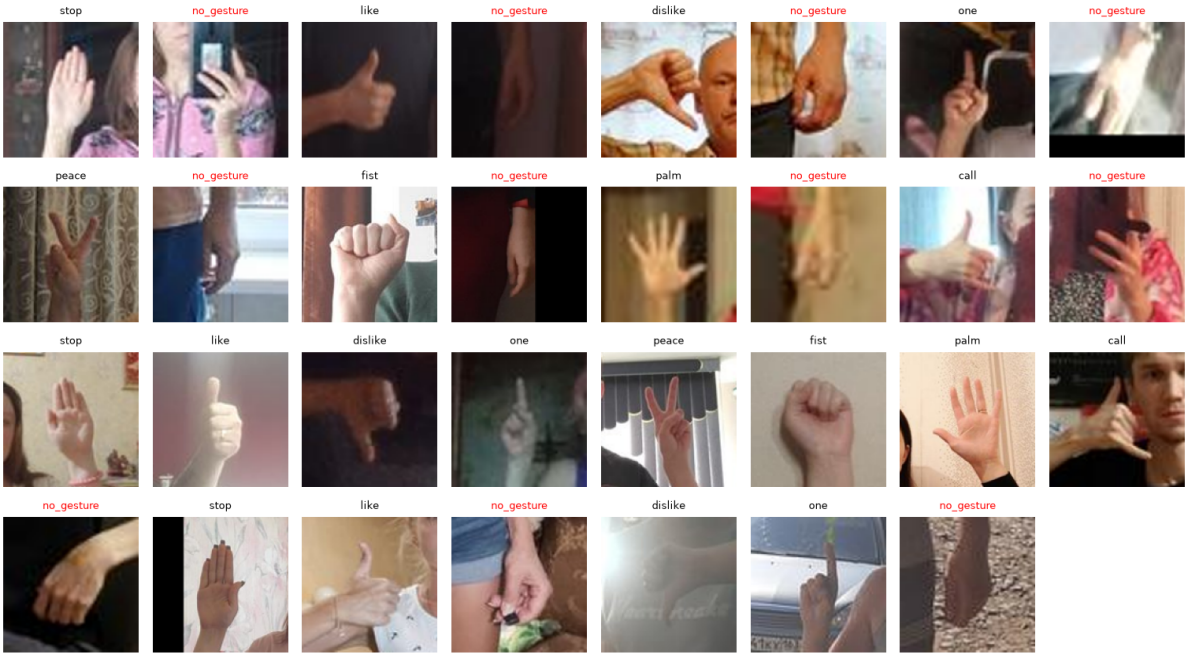



**Figura 2:** Recortes de 128 × 128 px de las nueve clases. Los ejemplos de *no_gesture* (en rojo) son manos en reposo, sujetando objetos o parcialmente visibles. Se aprecia la baja resolución de muchas manos, que se amplían al recortarlas.

que la entrada de 128 × 128 se reduce a mapas de 4 × 4. En los tres bloques más profundos se añade `Dropout2d` con probabilidades 0,1, 0,2 y 0,3: apaga mapas de características completos, lo que regulariza mejor que apagar píxeles sueltos, cuyos vecinos están muy correlacionados [10]. La cabeza aplica *global average pooling* [11], un *dropout* de 0,5 y una capa lineal de 256 a 9 salidas. El total es de 2,36 M parámetros, el 88 % de ellos en los dos últimos bloques, donde las convoluciones de 256 × 256 canales son las más caras. Gracias al *global average pooling*, la cabeza solo tiene 2 313 parámetros; una capa lineal sobre los 256 × 4 × 4 valores aplanados tendría 36 873, dieciséis veces más, y una capa oculta intermedia los multiplicaría de nuevo.

Las variantes de la ablación se obtienen con dos interruptores de la misma clase: *sin BN* sustituye cada capa BN por la identidad (las convoluciones conservan su sesgo) y *sin dropout* pone a cero todas las probabilidades de `Dropout2d` y `Dropout`. Así las variantes comparten exactamente el resto de la arquitectura. Para la búsqueda de hiperparámetros (apartado 4.5), la misma clase admite además otra lista de canales (que fija también el número de bloques), una capa oculta opcional en la cabeza y otra función de activación; con los valores por defecto la red es exactamente la descrita.

## 4.4. Entrenamiento

Todos los modelos se entrenan con entropía cruzada (sin pesos de clase, porque las clases están equilibradas) y Adam [20] con tasa de aprendizaje inicial $10^{-3}$, *weight decay* $10^{-4}$ y lotes de 64 imágenes, salvo el modelo final, que usa lotes de 32 (apartado 4.5). El último lote incompleto de cada época se descarta para no tener lotes de una imagen en BN. La tasa de aprendizaje se divide por dos cuando la pérdida de validación no mejora en 3 épocas (`ReduceLROnPlateau`). La parada anticipada [21] detiene el entrenamiento tras 6 épocas sin mejorar la pérdida de validación, con un máximo de 50 épocas, y restaura los pesos de la mejor época, que son los que se guardan en el *checkpoint*. Cada experimento fija la semilla (42 por defecto) de Python, NumPy y PyTorch; como algunas operaciones de cuDNN no son deterministas, dos ejecuciones con la misma semilla

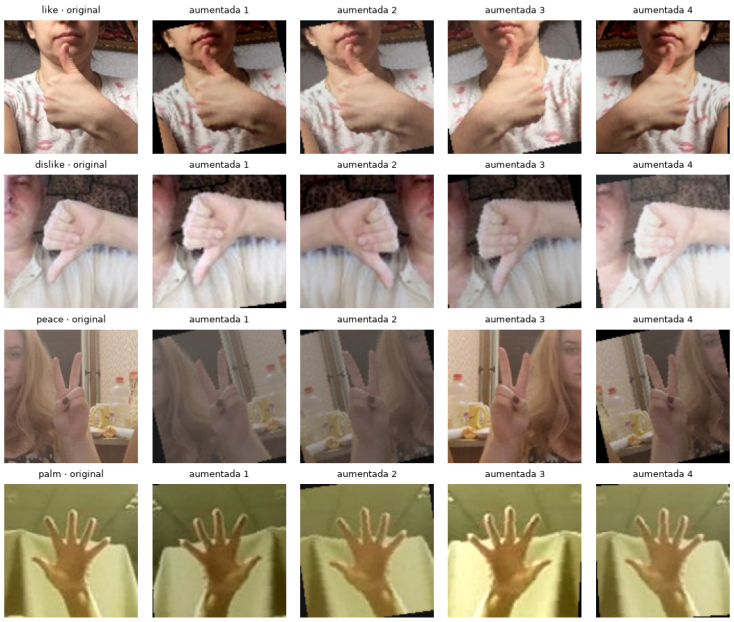



**Figura 3:** Aumento de datos: un recorte original por fila y cuatro versiones aumentadas. Las esquinas negras proceden de la rotación, igual que el relleno del recorte.

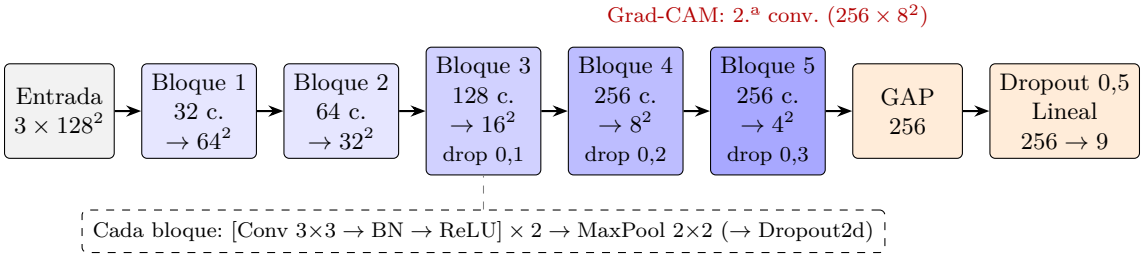



**Figura 4:** Arquitectura de GestureCNN (2,36 M parámetros). Bajo cada bloque se indica el número de canales, la resolución de salida y la probabilidad de `Dropout2d`. Grad-CAM se calcula sobre la segunda convolución del último bloque, antes del *pooling*.

pueden diferir ligeramente. Todos los entrenamientos se hicieron con PyTorch 2.14.1 en una GPU GTX 1660 Ti de portátil.

**Referencia: ResNet18.** Cargamos ResNet18 con los pesos de ImageNet de `torchvision`, sustituimos la capa final por una lineal de 512 a 9 salidas y reajustamos *toda* la red con los mismos datos, el mismo aumento y el mismo protocolo, salvo la tasa de aprendizaje inicial, diez veces menor ($10^{-4}$) para no destruir los rasgos preentrenados. La entrada es la misma de $128 \times 128$, normalizada con nuestras estadísticas y no con las de ImageNet; al reajustar todas las capas, la red se adapta a esa diferencia de escala.

## 4.5. Búsqueda de hiperparámetros

Partiendo del modelo completo (tasa de aprendizaje $10^{-3}$, lotes de 64, cinco bloques de 32 a 256 filtros, sin capa oculta en la cabeza y ReLU), hicimos una búsqueda manual en la que cada configuración cambia *un único* hiperparámetro y deja fijos todos los demás, incluidas la semilla, la partición y la parada anticipada. Probamos 9 configuraciones:

- **Tasa de aprendizaje**: $3 \cdot 10^{-4}$ y $3 \cdot 10^{-3}$, un factor de unas tres veces por debajo y por encima de la inicial.

- **Tamaño de lote**: 32 y 128. Con la misma tasa de aprendizaje, un lote menor da más actualizaciones por época y un gradiente más ruidoso.
- **Capacidad**: la mitad de filtros en cada bloque (de 16 a 128, 591 k parámetros); cuatro bloques en lugar de cinco (se quita el último, de modo que el mapa final es de $8 \times 8$; 1,18 M parámetros); y una capa oculta de 128 neuronas entre el *global average pooling* y la salida.
- **Activación**: ELU [22] y LeakyReLU con pendiente negativa 0,1, en lugar de ReLU.

Cada configuración se entrenó una vez y se evaluó solo en validación. El criterio de elección, fijado de antemano, es la exactitud de validación. Cambiar un factor cada vez permite atribuir cada diferencia a un solo hiperparámetro y cuesta una ejecución por valor probado (unos 20 minutos en nuestra GPU), mientras que una rejilla completa con estos valores necesitaría cientos. El precio es que no se exploran las interacciones entre hiperparámetros.

### 4.6. Protocolo de evaluación

**Selección del modelo.** Todas las decisiones se tomaron con validación, pero el modelo final se eligió en dos fases, y lo indicamos explícitamente porque afecta a la lectura del test. En la primera fase, el modelo final era la variante *completa* entrenada con la semilla 42. En la primera ablación, la única disponible cuando se tomó la decisión, obtuvo la menor pérdida y la mayor exactitud de validación (columna s42 de la tabla 5). Con él, con las variantes y con ResNet18 se evaluó el test una vez por modelo. En la segunda fase hicimos la búsqueda de hiperparámetros (apartado 4.5), solo con validación. La configuración con lotes de 32 obtuvo la mayor exactitud de validación y pasó a ser el modelo final. La decisión se tomó antes de evaluar esta configuración en test, que también se evaluó una sola vez. Aun así, cuando se tomó, el conjunto de test ya se había mirado con el modelo anterior. Por eso presentamos en test los dos modelos, el final y el completo, y los comparamos (apartado 5.3).

La segunda semilla de la ablación se entrenó antes de la búsqueda. Aunque en ella la variante sin *dropout* queda ligeramente por delante en validación, no la elegimos: la diferencia es del orden de la variación entre ejecuciones (apartado 5.4).

**Métricas.** Reportamos la exactitud, la F1 macro (media sin ponderar de las F1 por clase) y la precisión, *recall* y F1 de cada clase, además de la matriz de confusión normalizada por filas, cuya diagonal es el *recall*.

**Incertidumbre.** Las imágenes de una misma persona no son independientes: comparten mano, fondo e iluminación. Un intervalo de confianza calculado remuestreando imágenes sería, por tanto, demasiado estrecho. Por eso calculamos intervalos del 95 % por *bootstrap* de percentiles remuestreando *personas* con reemplazo (2.000 réplicas) [18].

**Comparación entre modelos.** Para decidir si dos modelos evaluados sobre las mismas imágenes de test difieren, usamos la prueba exacta de McNemar [16, 17]. La prueba solo tiene en cuenta las imágenes en las que *un único* modelo acierta: si la diferencia fuera ruido, esos desacuerdos se repartirían al 50 % entre los dos modelos. Repetimos la ablación con una segunda semilla (43) para estimar cuánto varía el resultado entre ejecuciones idénticas. La segunda semilla figuraba en el plan de trabajo como opcional y se ejecutó después de ver los resultados de validación de la primera ablación; las pruebas de McNemar se añadieron entonces. Ambos son, por tanto, análisis *exploratorios*, y las pruebas no se corrigen por comparaciones múltiples.

**Explicabilidad.** Calculamos Grad-CAM [15] sobre la última capa convolucional: el mapa de la clase $c$ es $\mathrm{ReLU}(\sum_k \alpha_k^c A^k)$, donde $A^k$ son los mapas de activación de la capa y $\alpha_k^c$ es la media

espacial del gradiente del *logit* de $c$ respecto a $A^k$. Usamos el *logit* y no la probabilidad porque la *softmax* se satura con confianzas altas y anula los gradientes. En GestureCNN la capa tiene una resolución de $8 \times 8$; en ResNet18, de $4 \times 4$. Los mapas se calculan siempre sobre validación.

**Coste.** Medimos el número de parámetros, las multiplicaciones-acumulaciones (MACs) por imagen de las capas convolucionales y lineales, el tiempo por época de entrenamiento y la latencia de inferencia con un lote de una imagen, que es el caso de la demo (un recorte por fotograma). La latencia es la mediana y el percentil 95 de 200 pasadas, tras 20 de calentamiento, en la CPU (Intel Core i7-9750H @ 2.60GHz, 6 hilos) y en la GPU.

# 5. Resultados

## 5.1. Búsqueda de hiperparámetros

La tabla 2 recoge las 9 configuraciones de la búsqueda junto con la de referencia, y la figura 5 compara sus curvas de validación agrupadas por tipo de hiperparámetro. El primer resultado es que la red es poco sensible a todos los cambios probados: salvo el lote de 32, todas las configuraciones quedan entre el 98,74 % y el 99,07 % de exactitud de validación, a pocas imágenes de la referencia (98,99 %, 24 errores). Como referencia del ruido, dos ejecuciones idénticas del modelo completo con semillas distintas obtienen 98,99 % y 98,82 % (24 y 28 errores), así que la mayoría de las diferencias de la tabla no deben interpretarse como efectos de los hiperparámetros.

Por grupos:

- **Tasa de aprendizaje.** Con $3 \cdot 10^{-3}$ y con $3 \cdot 10^{-4}$ la parada anticipada actúa antes (épocas 29 y 31, frente a 44), pero la exactitud final no cambia de forma apreciable. Probablemente, Adam y la reducción de la tasa en las mesetas hacen que el valor inicial importe poco dentro de este intervalo.
- **Tamaño de lote.** Es el único cambio que mejora tanto la exactitud como la pérdida de validación: con lotes de 32 se obtiene la menor pérdida de la búsqueda y 17 errores, frente a 24. Con la misma tasa de aprendizaje, un lote menor da el doble de actualizaciones por época y un gradiente más ruidoso, que actúa como un regularizador adicional. El lote de 128 va en la dirección contraria: para antes y queda ligeramente por debajo.
- **Capacidad.** Con la mitad de filtros en cada bloque, la red tiene cuatro veces menos parámetros (591 k) y obtiene la misma exactitud (99,03 %). Esto indica que GestureCNN está sobredimensionada para esta tarea. Quitar el último bloque es el cambio que más empeora el resultado y el que más ralentiza el aprendizaje (figura 5): con un mapa final de $8 \times 8$, cada filtro de la última capa ve una región menor de la mano antes del *global average pooling*. Añadir una capa oculta a la cabeza no aporta nada.
- **Activación.** ELU y LeakyReLU quedan algo por debajo de ReLU y tardan más en converger. Con ELU, el entrenamiento llegó al máximo de 50 épocas.

Siguiendo el criterio fijado de antemano, elegimos como modelo final la configuración con lotes de 32. Su ventaja en validación (siete errores menos que la referencia) es mayor que la diferencia entre las dos semillas del modelo completo, pero procede de una sola ejecución. Además, al elegir la mejor de diez configuraciones con el mismo conjunto de validación, su exactitud de validación tiende a sobrestimar la real. El test permite comprobarlo (apartado 5.3).

**Tabla 2:** Búsqueda manual de hiperparámetros, solo en validación (2.375 imágenes). Cada fila cambia un único hiperparámetro de la referencia (tasa de aprendizaje $10^{-3}$, lote de 64, cinco bloques de 32 a 256 filtros, sin capa oculta y ReLU); todas usan la semilla 42. Pérdida: pérdida de validación en la mejor época. En negrita, la configuración elegida como modelo final.

| Hiperparámetro | Valor | Parám. | Épocas (mejor) | Pérdida | Acc. | F1 macro | Errores |
|---|---|---|---|---|---|---|---|
| Referencia (*completo*) | — | 2,36 M | 44 (38) | 0,0358 | 98,99 % | 98,99 % | 24 |
| Tasa de aprendizaje | $3 \cdot 10^{-4}$ | 2,36 M | 31 (25) | 0,0329 | 98,86 % | 98,86 % | 27 |
| | $3 \cdot 10^{-3}$ | 2,36 M | 29 (23) | 0,0357 | 99,07 % | 99,07 % | 22 |
| Tamaño de lote | 32 | **2,36 M** | **45 (39)** | **0,0280** | **99,28 %** | **99,28 %** | **17** |
| | 128 | 2,36 M | 33 (27) | 0,0391 | 98,91 % | 98,91 % | 26 |
| Filtros | 16–128 | 0,59 M | 41 (35) | 0,0368 | 99,03 % | 99,03 % | 23 |
| Nº de bloques | 4 | 1,18 M | 39 (33) | 0,0500 | 98,74 % | 98,73 % | 30 |
| Capa oculta | 128 neuronas | 2,39 M | 40 (34) | 0,0368 | 98,91 % | 98,91 % | 26 |
| Activación | ELU | 2,36 M | 50 (46) | 0,0414 | 98,82 % | 98,82 % | 28 |
| | LeakyReLU | 2,36 M | 49 (43) | 0,0319 | 98,78 % | 98,78 % | 29 |

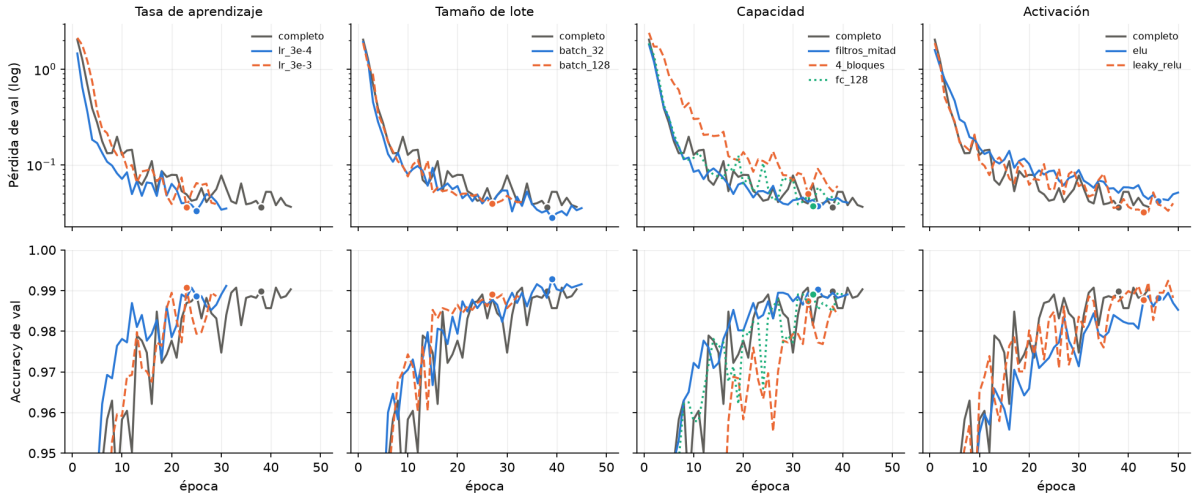



**Figura 5:** Curvas de validación de la búsqueda, un panel por tipo de hiperparámetro. En gris, la configuración de referencia. Arriba, la pérdida en escala logarítmica; abajo, la exactitud, ampliada por encima del 95 %. El punto marca la mejor época (pesos restaurados).

## 5.2. Entrenamiento del modelo final

La figura 6 muestra las curvas de entrenamiento del modelo final, GestureCNN con lotes de 32. La parada anticipada detuvo el entrenamiento en la época 45 y restauró los pesos de la época 39, la de menor pérdida de validación. La exactitud de validación supera el 95 % en la época 6; a partir de ahí, las 3 reducciones de la tasa de aprendizaje (épocas 21, 36 y 44) producen las mejoras finales.

Las curvas de entrenamiento y validación casi se superponen: al final del entrenamiento la exactitud de entrenamiento supera a la de validación en solo 0,6 puntos, de modo que no hay sobreajuste apreciable. En las primeras épocas la exactitud de validación es incluso *mayor* que la de entrenamiento. No es una anomalía: las métricas de entrenamiento se miden con el *dropout* activo y sobre imágenes aumentadas, que son más difíciles que las de validación.

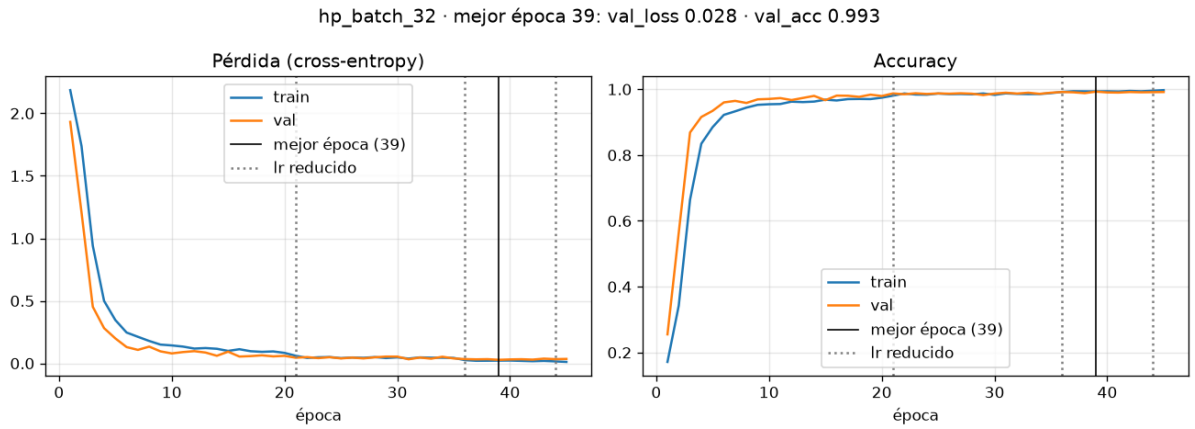



**Figura 6:** Pérdida y exactitud de entrenamiento y validación del modelo final. Línea continua: mejor época (pesos restaurados). Líneas punteadas: reducciones de la tasa de aprendizaje.

**Tabla 3:** Métricas por clase del modelo final en test. El *recall* es la diagonal de la matriz de confusión de la figura 7.

| Clase | Orden | Precisión | Recall | F1 | $n$ |
|---|---|---|---|---|---|
| *stop* | Parar | 97,4 % | 98,9 % | 98,1 % | 262 |
| *like* | Avanzar | 99,6 % | 98,8 % | 99,2 % | 260 |
| *dislike* | Retroceder | 99,6 % | 98,9 % | 99,2 % | 267 |
| *one* | Girar a la izquierda | 99,6 % | 99,6 % | 99,6 % | 266 |
| *peace* | Girar a la derecha | 98,5 % | 99,2 % | 98,9 % | 266 |
| *fist* | Cerrar pinza | 100,0 % | 100,0 % | 100,0 % | 261 |
| *palm* | Abrir pinza | 98,9 % | 97,7 % | 98,3 % | 264 |
| *call* | Volver al inicio | 98,9 % | 100,0 % | 99,4 % | 264 |
| *no_gesture* | Ninguna (quieto) | 98,9 % | 98,1 % | 98,5 % | 264 |

## 5.3. Rendimiento en test

Evaluado una sola vez sobre las 2.374 imágenes de las 726 personas de test, el modelo final alcanza una exactitud del 99,03 % (IC del 95 %: [98,61 %; 99,42 %]) y una F1 macro del 99,03 % (IC: [98,61 %; 99,42 %]), con 23 errores. El objetivo del 85 % se supera con un margen de unos 14 puntos, y el extremo inferior del intervalo de confianza queda también muy por encima.

**Comparación con el modelo final anterior.** Como se explica en la apartado 4.6, el test ya se había evaluado con el modelo completo (lotes de 64) antes de la búsqueda. Ese modelo obtuvo un 98,99 % (IC del 95 %: [98,53 %; 99,40 %]) con 24 errores, de modo que en test los dos modelos son prácticamente iguales. Sobre las mismas imágenes, 9 las acierta solo el modelo final y 8 solo el completo (McNemar: $p = 1{,}000$). La mejora de validación que motivó el cambio no se traslada al test. Es lo esperable si se debía en buena parte al ruido y a haber elegido la mejor de diez configuraciones: la exactitud de validación del modelo final (99,28 %) supera a la de test, mientras que la del modelo completo, que no se eligió entre tantas alternativas, coincidía con ella (98,99 % y 98,99 %). Ninguna de las conclusiones del trabajo depende de cuál de los dos modelos se tome como final.

El rendimiento es homogéneo entre clases (tabla 3): todas superan el 98 % de F1, y la peor es *stop* (98,1 %). Las confusiones más frecuentes (figura 7) son *palm*→*stop* (6), *stop*→*palm* (3), *dislike*→*no_gesture* (2), *like*→*call* (2). De los 23 errores, 11 corresponden a los dos pares de gestos visualmente próximos que anticipamos en la apartado 3: *stop*/*palm*, que solo se distinguen por si los dedos están juntos o separados, y *like*/*call*, que comparten el pulgar extendido y se

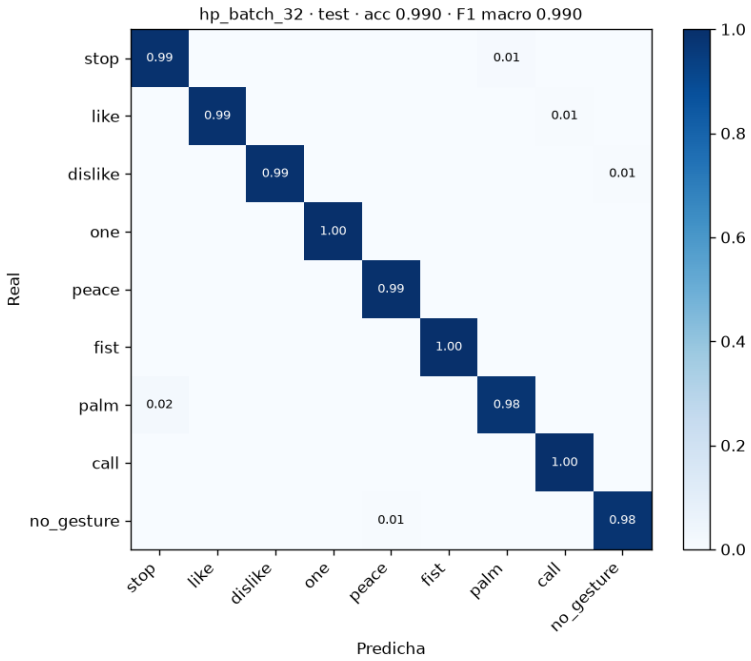



**Figura 7:** Matriz de confusión del modelo final en test, normalizada por filas (cada fila suma 1; la diagonal es el *recall*). Se omiten las celdas por debajo de 0,005.

**Tabla 4:** Estudio de ablación: media de las semillas 42 y 43 de cada variante. Cada variante elimina un único componente del modelo completo; el resto de la configuración es idéntico. La columna de épocas indica las épocas entrenadas con cada semilla (máximo 50). †ResNet18 se entrenó una sola vez, con la semilla 42.

| | | | | Validación | | Test | | |
|---|---|---|---|---|---|---|---|---|
| Variante | Dropout | BN | Augm. | Acc. | F1 macro | Acc. | F1 macro | Épocas |
| Completo | sí | sí | sí | 98,91 % | 98,91 % | 99,01 % | 99,01 % | 44 / 34 |
| Sin dropout | no | sí | sí | 99,03 % | 99,03 % | 98,97 % | 98,97 % | 29 / 50 |
| Sin BN | sí | no | sí | 97,56 % | 97,55 % | 97,81 % | 97,80 % | 50 / 50 |
| Sin augmentation | sí | sí | no | 98,74 % | 98,74 % | 98,65 % | 98,65 % | 23 / 24 |
| ResNet18 preentrenado† | — | sí | sí | 99,41 % | 99,41 % | 99,20 % | 99,20 % | 23 |

diferencian por el meñique.

Desde el punto de vista del robot, el error más grave es interpretar una mano en reposo como una orden. En test, 5 de los 264 ejemplos de *no_gesture* (1,9 %) se clasificaron como algún gesto, y 3 gestos se clasificaron como *no_gesture*. Estas tasas se refieren a fotogramas aislados; el filtro temporal de la demo (apartado 6) está pensado precisamente para que un error aislado no llegue a mover el robot.

## 5.4. Estudio de ablación

La tabla 4 resume las cuatro variantes de GestureCNN, cada una entrenada con dos semillas, junto con ResNet18. La tabla 5 detalla cada ejecución y las pruebas de McNemar, y la figura 8 compara las curvas de validación.

**Normalización por lotes.** Quitar BN es la única ablación con un efecto claro y consistente. La variante sin BN obtiene la peor exactitud con las dos semillas (media de 97,81 % en test

**Tabla 5:** Detalle por semilla. Exactitud de validación y número de errores en test (de 2.374 imágenes) de cada ejecución. Las tres últimas columnas comparan en test cada variante (semilla 42) con el modelo completo (semilla 42): imágenes que solo acierta el completo, que solo acierta la variante, y valor $p$ de la prueba exacta de McNemar.

| | Acc. val | | Errores test | | McNemar vs. completo (semilla 42) | | |
|---|---|---|---|---|---|---|---|
| Variante | s42 | s43 | s42 | s43 | solo compl. | solo var. | $p$ |
| Completo | 98,99 % | 98,82 % | 24 | 23 | — | — | — |
| Sin dropout | 98,95 % | 99,12 % | 28 | 21 | 14 | 10 | 0,541 |
| Sin BN | 97,52 % | 97,60 % | 55 | 49 | 42 | 11 | $<0{,}001$ |
| Sin augmentation | 98,65 % | 98,82 % | 27 | 37 | 15 | 12 | 0,701 |

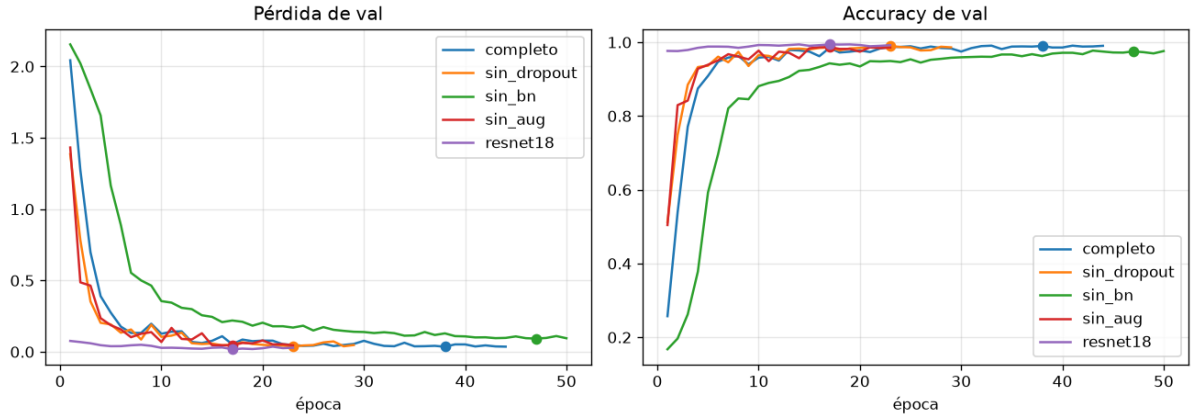



**Figura 8:** Pérdida y exactitud de validación de las cuatro variantes (semilla 42) y de ResNet18. El punto marca la mejor época de cada modelo.

frente a 99,01 % del modelo completo) y comete más del doble de errores en test. La diferencia es significativa: de las imágenes en las que solo uno de los dos modelos acierta, 42 las acierta el completo y 11 la variante sin BN ($p < 0{,}001$). Además, sin BN el aprendizaje es mucho más lento (figura 8): tras la primera época la exactitud de validación es del 16,8 %, poco por encima del 11,1 % de un clasificador aleatorio, y no supera el 95 % hasta la época 25, frente a la época 7 del modelo completo. Con las dos semillas el entrenamiento agotó las 50 épocas sin que actuara la parada anticipada, y la mejor época fue una de las últimas (47 y 49). Por tanto, el resultado debe leerse como *«con el mismo presupuesto de épocas, sin BN se llega a un modelo peor»*: con más épocas, o con una tasa de aprendizaje ajustada para esta variante, la distancia probablemente se reduciría.

**Dropout.** Quitar el *dropout* no tiene un efecto que podamos distinguir de la variación entre ejecuciones. Con la semilla 42 el modelo completo es ligeramente mejor, y con la semilla 43 lo es la variante sin *dropout*; en media, las exactitudes de test son prácticamente iguales (99,01 % frente a 98,97 %), con una diferencia del mismo orden que la que separa dos ejecuciones idénticas del modelo completo, y la prueba de McNemar no detecta diferencias ($p = 0{,}541$). El *dropout* sí deja una huella en las curvas: sin él, la red aprende más deprisa al principio (51,2 % tras la primera época frente al 25,7 %) y la diferencia final entre la exactitud de entrenamiento y la de validación es mayor (1,1 puntos frente a 0,5). Es decir, el *dropout* reduce el ajuste a los datos de entrenamiento, pero con 11.088 imágenes y aumento de datos el modelo completo apenas sobreajusta, y esa regularización adicional no se traduce en más exactitud.

**Aumento de datos.** Sin aumento de datos, la exactitud media en test baja a 98,65 % y los errores en test pasan de 24 a 27 y a 37 con las dos semillas, frente a 24 y 23 del modelo completo. La dirección del efecto es la esperada y se repite en test con ambas semillas, pero es pequeña: en validación, con la semilla 43, ambas variantes empatan, y la prueba de McNemar con la semilla 42 no declara significativa la diferencia ($p = 0{,}701$). Donde el efecto se ve con claridad es en la dinámica del entrenamiento: sin aumento de datos la red memoriza el conjunto de entrenamiento (su exactitud de entrenamiento llega al 99,9 % con ambas semillas), la pérdida de validación deja de mejorar antes y la parada anticipada actúa en la época 23 y 24, las más tempranas de todas las variantes. La diferencia final entre entrenamiento y validación, de 1,3 puntos, es la mayor de la ablación. Es la firma típica de un sobreajuste incipiente que la parada anticipada corta a tiempo.

**Variación entre semillas.** Dos ejecuciones idénticas del modelo completo difieren en 24 y 23 errores de test y en la época de parada (44 y 34). Diferencias entre variantes de ese orden, de unas pocas imágenes, no deben interpretarse como efectos de la arquitectura. Este es el principal motivo por el que solo consideramos concluyente el efecto de BN.

### 5.5. Comparación con ResNet18

ResNet18 preentrenado parte de una ventaja clara: tras la primera época ya alcanza un 97,6 % de exactitud en validación, frente al 25,5 % de GestureCNN, que parte de pesos aleatorios. La parada anticipada lo detuvo en la época 23 (mejor época: 17). En test obtiene una exactitud del 99,20 % (IC del 95 %: [98,76 %; 99,57 %]) y una F1 macro del 99,20 %, con 19 errores frente a los 23 de GestureCNN.

La diferencia, sin embargo, no es estadísticamente significativa. Sobre las mismas imágenes de test, solo 15 imágenes las acierta ResNet18 y falla GestureCNN, y 11 al revés; la prueba de McNemar da $p = 0{,}557$, y los intervalos de confianza de ambos modelos se solapan ampliamente. Con estos datos, por tanto, no podemos afirmar que un modelo con 4,7 veces más parámetros y preentrenado con más de un millón de imágenes generalice mejor que nuestra red entrenada desde cero. Lo que sí aporta el preentrenamiento es una convergencia mucho más rápida.

Las confusiones de ResNet18 son del mismo tipo: su error más frecuente también es el par *palm*/*stop*. Sus mapas Grad-CAM (figura 9) se centran igualmente en la mano, pero son mucho más gruesos, porque su última capa convolucional tiene una resolución de $4 \times 4$ frente a $8 \times 8$ en GestureCNN; para explicar decisiones a nivel de dedos, la red propia resulta más interpretable.

### 5.6. Explicabilidad con Grad-CAM

La figura 10 muestra un acierto de cada clase en validación con su mapa Grad-CAM. En todos los casos la activación se concentra sobre la mano y no sobre el fondo: en *stop* la cara de la persona aparece en el recorte y queda fuera del mapa. Además, los mapas señalan los rasgos que definen cada gesto: la palma y la base de los dedos en *stop* y *palm*, el puño con el pulgar hacia abajo en *dislike* y, sobre todo, *call*, cuyo mapa tiene dos máximos, uno sobre el pulgar y otro sobre el meñique, los dos dedos extendidos que definen el gesto. El ejemplo con menos confianza es un *peace* (0,57) con el fondo partido por una franja negra de relleno, en el que el mapa se reparte entre los dos dedos y el borde de la imagen.

La figura 11 muestra los errores en los que el modelo estaba más seguro. Son especialmente informativos porque se agrupan en tres tipos:

- **Ruido de etiqueta.** El primer error es un *no_gesture* que la red clasifica como *palm* con probabilidad 1,00, y la imagen muestra, efectivamente, una palma abierta hacia la cámara;

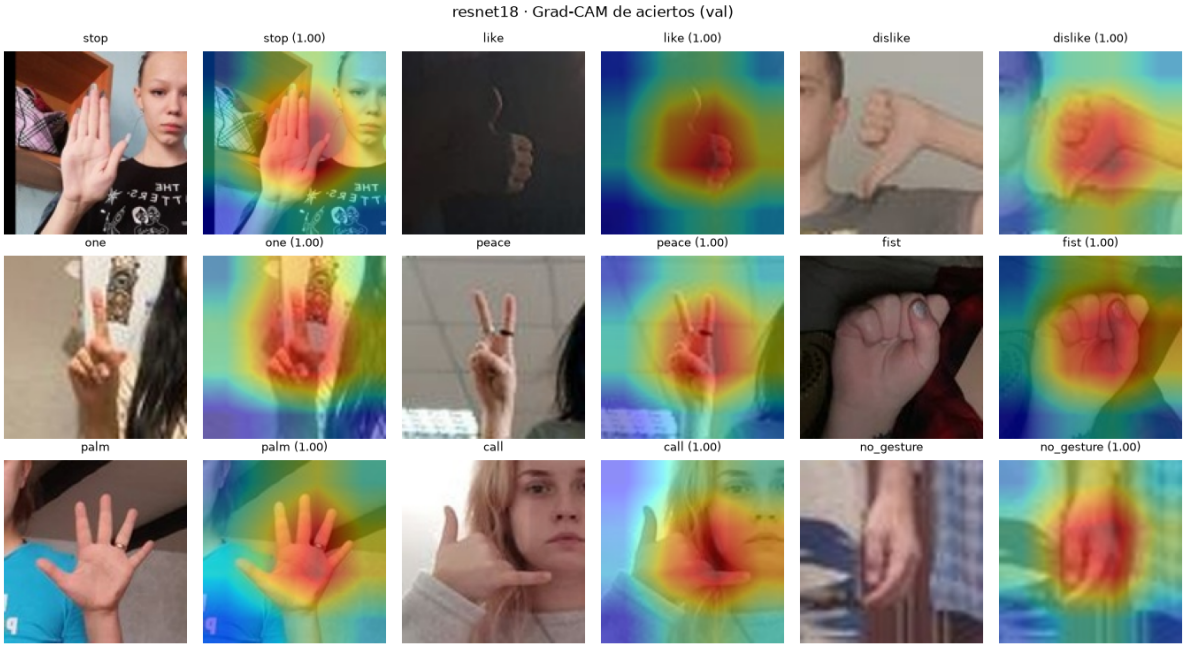



**Figura 9:** Grad-CAM de ResNet18 sobre un acierto de cada clase (validación). Los mapas se calculan sobre una rejilla de $4 \times 4$ y localizan la mano con menos detalle que los de GestureCNN (figura 10).

la etiqueta *no_gesture* procede de la anotación original. En este caso la predicción es más razonable que la etiqueta.

- **Objetos con forma de gesto.** Un *no_gesture* que sujeta un teléfono se clasifica como *peace*, y en un *one* delante de una estantería el borde vertical del mueble, junto al índice, se lee como un segundo dedo extendido (*peace*).
- **Pares próximos e imágenes de baja calidad.** En los dos *palm*→*stop* la separación entre los dedos es intermedia, y los mapas de la clase real y de la predicha se concentran en la misma zona: la red mira el lugar correcto, pero la evidencia es ambigua. Los *like* vistos de perfil se confunden con *call* o con *one*, porque en esa vista el pulgar levantado se parece al meñique de *call* o al índice de *one*. Un *palm* pixelado hasta ser irreconocible se clasifica como *peace*.

## 5.7. Coste computacional

La tabla 6 resume el coste de los dos modelos. El resultado más llamativo es que el número de parámetros no predice el coste de cálculo: GestureCNN tiene 4,7 veces *menos* parámetros que ResNet18, pero necesita 1,6 veces *más* operaciones por imagen (920 M frente a 592 M de MACs). La causa está en la resolución a la que trabajan las primeras capas. GestureCNN aplica sus dos primeros bloques a $128 \times 128$ y $64 \times 64$ px; estos bloques acumulan el 43 % de las operaciones con solo el 3 % de los parámetros. ResNet18, en cambio, reduce la resolución a la cuarta parte en sus dos primeras capas (convolución $7 \times 7$ con paso 2 y *max pooling*). Esto explica que una época de GestureCNN tarde 2,1 veces más que una de ResNet18 (25,5 s frente a 12,1 s, ambas con lotes de 64; con los lotes de 32 del modelo final, 26,8 s).

Para la aplicación, sin embargo, lo relevante es la latencia con una sola imagen, y ahí la relación se invierte: GestureCNN clasifica un recorte en 8,81 ms de mediana en la CPU del portátil y en 1,04 ms en la GPU, frente a 9,70 ms y 2,06 ms de ResNet18. Con lotes de 64 imágenes el tiempo depende sobre todo del número de operaciones, pero con una sola imagen pesa más el coste fijo de lanzar cada capa, y ResNet18 tiene el doble de convoluciones (20 frente a 10). En

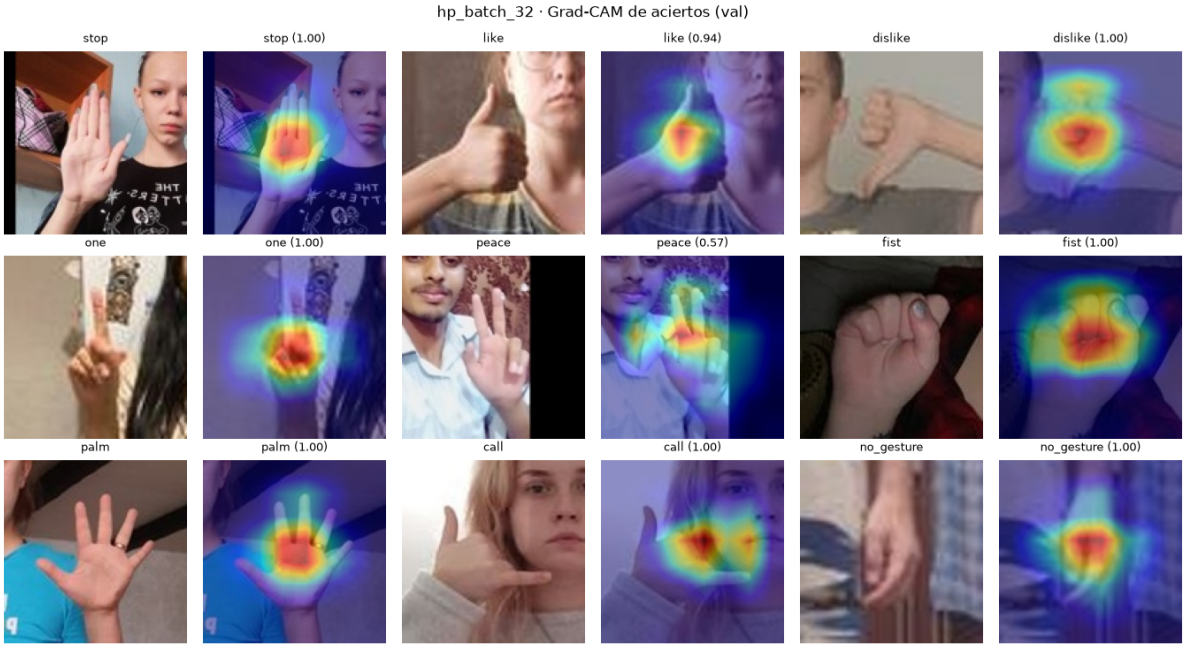



**Figura 10:** Grad-CAM del modelo final sobre un acierto de cada clase (validación, elegidos al azar con semilla fija). Entre paréntesis, la probabilidad asignada a la clase. Rojo: regiones que empujan la predicción hacia esa clase.

**Tabla 6:** Coste de GestureCNN y ResNet18 con entradas de $128 \times 128$. MACs: multiplicaciones-acumulaciones por imagen. s/época: con lotes de 64 en ambos modelos. Latencia con un lote de una imagen (mediana y percentil 95 de 200 pasadas). Equipo: Intel Core i7-9750H @ 2.60GHz y GTX 1660 Ti.

| | | | | CPU (ms) | | GPU (ms) | |
|---|---|---|---|---|---|---|---|
| Modelo | Parámetros | MACs | s/época | mediana | p95 | mediana | p95 |
| GestureCNN | 2,36 M | 920 M | 25,5 | 8,81 | 11,04 | 1,04 | 1,14 |
| ResNet18 | 11,18 M | 592 M | 12,1 | 9,70 | 11,74 | 2,06 | 2,46 |

cualquier caso, ambos modelos caben holgadamente en los 33 ms por fotograma de una cámara a 30 fps, incluso sin GPU, de modo que el clasificador no será el cuello de botella de la demo. Si hiciera falta reducir el coste, la primera medida sería añadir paso 2 a la primera convolución de GestureCNN, que es donde se concentra el cálculo.

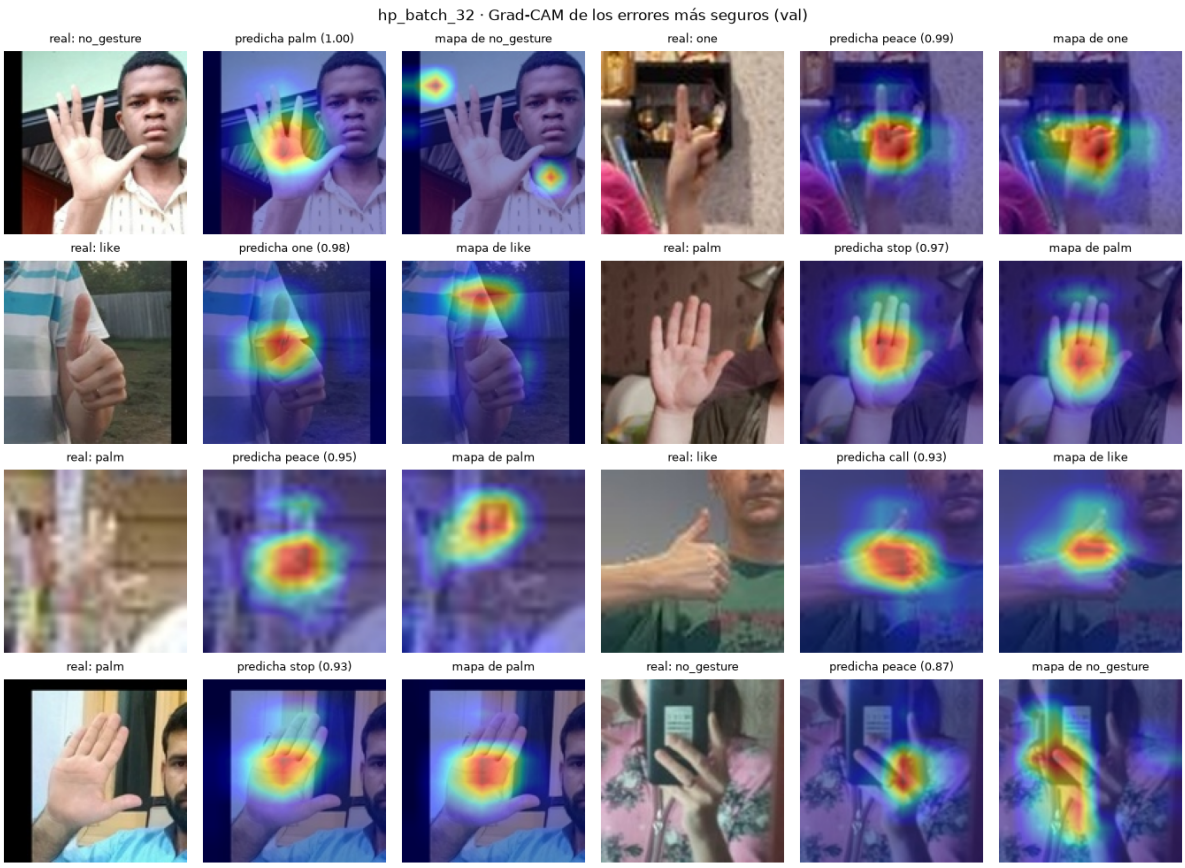



**Figura 11:** Errores más seguros del modelo final en validación. Para cada uno: la imagen con su clase real, el mapa de la clase predicha y el mapa de la clase real.

# 6. Demo en tiempo real con un robot simulado

Esta sección describe cómo se conecta el clasificador a un robot simulado en ROS 2 [23] con *turtlesim*. Usamos *turtlesim* porque recibe las órdenes de velocidad con la misma interfaz que los robots móviles reales (mensajes `Twist` en un *topic* `cmd_vel`), así que el paso a un simulador como Gazebo, o a un robot físico, no cambia el nodo de control.

## 6.1. Diseño

El sistema procesa cada fotograma de la cámara en cinco etapas (figura 12):

1. **Captura** del fotograma con OpenCV.
2. **Detección de la mano** con MediaPipe Hands [6]: la caja se obtiene como el rectángulo que envuelve los 21 puntos clave de la mano.
3. **Recorte** con la misma función `crop_hand` que se usó para generar los datos de entrenamiento (cuadrado, margen del 25 %, $128 \times 128$ px) y normalización con las estadísticas de entrenamiento. Compartir esta función evita una fuente silenciosa de discrepancia entre entrenamiento e inferencia.
4. **Clasificación** con GestureCNN, que devuelve una probabilidad por clase.
5. **Filtrado y control**: una orden solo se emite si la probabilidad máxima supera 0,7 $y$ es la clase mayoritaria en una ventana de los últimos 5–7 fotogramas. Si no se cumple, si no

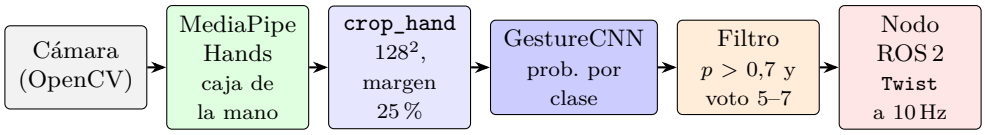



**Figura 12:** Etapas de la demo en tiempo real. Las etapas 3 y 4 reutilizan exactamente el preprocesado y el modelo evaluados en este trabajo.

**Tabla 7:** Correspondencia entre gestos y órdenes para *turtlesim*. Las órdenes de pinza no tienen equivalente directo en *turtlesim*.

| Gesto | Orden | Acción en *turtlesim* |
|---|---|---|
| *like* | Avanzar | `linear.x` $> 0$ |
| *dislike* | Retroceder | `linear.x` $< 0$ |
| *one* | Girar a la izquierda | `angular.z` $> 0$ |
| *peace* | Girar a la derecha | `angular.z` $< 0$ |
| *stop* | Parar | velocidad cero |
| *fist*, *palm* | Cerrar / abrir pinza | sin equivalente (ver texto) |
| *call* | Volver al inicio | servicio `teleport_absolute` |
| *no_gesture* | Ninguna | velocidad cero |

se detecta ninguna mano o si la clase es *no_gesture*, el nodo publica velocidad cero. Las velocidades se publican a 10 Hz en `/turtle1/cmd_vel`.

El filtro de la última etapa responde al segundo requisito de la apartado 1: un fotograma aislado mal clasificado, por ejemplo durante la transición entre dos gestos, no debe mover el robot. El umbral y la ventana intercambian seguridad por tiempo de reacción: a 30 fotogramas por segundo, una ventana de 7 fotogramas añade como mucho unos 0,2 s de retardo.

**Pendiente (Carlos):** indicar qué acción se asignó finalmente a *fist* y *palm* en *turtlesim* (por ejemplo, subir y bajar el lápiz con el servicio `set_pen`) y confirmar los valores finales del umbral, de la ventana de voto y de las velocidades lineal y angular.

### 6.2. Implementación y resultados

**Pendiente (Carlos):** describir la implementación del nodo `gesture_controller` (versión de ROS 2, entorno: Ubuntu nativo, máquina virtual o Docker) y cómo se integró el reconocedor con MediaPipe.

**Pendiente (Carlos):** tabla de latencias por etapa (captura, MediaPipe, recorte, CNN, filtro y extremo a extremo), con la media y el percentil 95 medidos con `time.perf_counter` sobre al menos unos cientos de fotogramas, indicando el equipo usado. Como referencia, la etapa de clasificación por sí sola tarda 8,81 ms en CPU y 1,04 ms en GPU (mediana, tabla 6).

**Pendiente (Carlos):** resultados cualitativos de la demo: si la tortuga obedece a cada gesto, activaciones falsas observadas (sobre todo durante las transiciones entre gestos), efecto del filtro y enlace al vídeo de la demostración.

## 7. Discusión

**¿Es creíble una exactitud del 99 %?** Una exactitud tan alta debe tratarse con cautela, porque la causa más habitual de resultados demasiado buenos es una fuga de información entre entrenamiento y test [2]. Nuestro plan de trabajo consideraba de hecho un resultado por encima del 98 % como una señal de alarma. Hemos descartado las fugas más probables: (i) la partición

es por persona y una prueba automática confirma que no hay personas compartidas; (II) las estadísticas de normalización se calculan solo con entrenamiento; (III) todas las decisiones se tomaron con validación y el test se evaluó una sola vez por modelo (con la salvedad, explicada en la apartado 4.6, de que el modelo final se cambió tras la búsqueda de hiperparámetros, cuando el test ya se había evaluado con el anterior); y (IV) validación y test, con personas distintas, dan resultados casi idénticos. Creemos que la exactitud alta se explica más bien por la naturaleza de la tarea tal como la hemos planteado. Los recortes se obtienen de las cajas anotadas a mano, así que la mano siempre está centrada y completa. Los gestos de HaGRID están posados deliberadamente para la cámara, y la mayoría de las clases tienen formas claramente distintas. La dificultad residual se concentra en *stop/palm* y *like/call* y en las etiquetas ruidosas de *no_gesture*.

Esta interpretación delimita también el alcance del resultado. La cifra del 99 % estima el rendimiento del *clasificador* sobre recortes de calidad HaGRID con cajas perfectas, no el del sistema completo con una webcam. En la demo, las cajas vendrán de MediaPipe y estarán peor centradas, la cámara y la iluminación serán otras y aparecerán posturas de transición entre gestos que HaGRID no contiene. Es esperable una caída de rendimiento por este cambio de dominio que nuestro experimento no cuantifica.

**Qué aporta cada componente.** La ablación muestra un resultado menos uniforme de lo que se suele esperar: de los tres componentes, solo la normalización por lotes tiene un efecto claro sobre la exactitud final. Su papel es sobre todo de *optimización*: sin BN la red aprende mucho más despacio y no llega a converger en 50 épocas, en línea con la motivación original de BN como técnica para estabilizar y acelerar el entrenamiento [8]. El *dropout* y el aumento de datos son, en cambio, técnicas de *regularización*, y su efecto se ve en la dinámica del entrenamiento (más distancia entre entrenamiento y validación y parada más temprana sin ellas) más que en la exactitud final. Nuestra interpretación es que, en este problema, el modelo completo sobreajusta poco: tiene muchos ejemplos por clase, clases equilibradas y la parada anticipada ya lo protege. No es un argumento para eliminar el *dropout* o el aumento de datos. Ambos deberían ganar importancia ante el cambio de dominio a la webcam (otras cámaras, iluminación y encuadres), que nuestro conjunto de test no representa; el aumento de datos se diseñó precisamente para simular esas variaciones.

**Sensibilidad a los hiperparámetros.** La búsqueda muestra que, alrededor de la configuración inicial, el resultado depende poco de los hiperparámetros: nueve cambios de tasa de aprendizaje, lote, capacidad y activación mueven la exactitud de validación menos de medio punto, y casi siempre dentro de la variación entre semillas. El único cambio elegido, el lote de 32, mejoró la validación pero no el test, lo que ilustra el riesgo de elegir entre muchas configuraciones con un único conjunto de validación pequeño. El resultado más útil de la búsqueda es otro: con cuatro veces menos parámetros la red rinde igual, así que la capacidad de GestureCNN no es lo que limita la exactitud.

**CNN propia frente a modelo preentrenado.** La comparación con ResNet18 matiza la idea de que un modelo más grande y preentrenado es siempre mejor. Con 11.088 imágenes de entrenamiento, una tarea bien acotada y entradas de $128 \times 128$, una red de 2,36 M parámetros entrenada desde cero queda a una distancia que no podemos distinguir del ruido estadístico. El preentrenamiento sí acelera mucho la convergencia, y podría marcar más diferencia con menos datos o en el cambio de dominio a la webcam, algo que no hemos medido. Por otra parte, el análisis de coste muestra que el número de parámetros es una mala medida del coste de inferencia: la red pequeña es la que más operaciones realiza, porque trabaja a mayor resolución en sus primeras capas.

**Limitaciones.**

- **Cajas anotadas.** Todo el entrenamiento y la evaluación usan las cajas de HaGRID. No hemos medido el rendimiento con las cajas de un detector ni con vídeo propio, que es el escenario real de uso.
- **Resolución.** Al trabajar con la versión de 384 px, la mayoría de las manos se amplían para llegar a $128 \times 128$ (mediana del lado de la caja: 68 px), lo que elimina detalle fino como la posición exacta del meñique, relevante en *like*/*call*.
- **Pocas repeticiones.** Dos semillas por variante bastan para ver la magnitud de la variación entre ejecuciones, pero no para estimar con precisión diferencias de pocas décimas. La ResNet18 se entrenó una sola vez.
- **Búsqueda de hiperparámetros limitada.** La búsqueda cambia un hiperparámetro cada vez, con una sola semilla por configuración, de modo que no explora interacciones y sus diferencias son del orden del ruido entre ejecuciones. Además, se hizo solo sobre el modelo completo: es posible que otra tasa de aprendizaje mejore alguna variante de la ablación, en particular la red sin BN.
- **Ruido de etiqueta en la clase nula.** Algunos *no_gesture* son en realidad gestos (apartado 5.6), lo que penaliza al modelo por predicciones correctas y hace que la exactitud medida sea una cota algo pesimista en esa clase.
- **Una sola fuente de datos.** Las personas, los fondos y la forma de grabar son los de HaGRID; no sabemos cómo se comporta el modelo con otras poblaciones o cámaras.

# 8. Conclusiones y trabajo futuro

Hemos mostrado que una red convolucional pequeña, entrenada desde cero, reconoce un vocabulario de ocho gestos más una clase nula con una exactitud del 99,03 % (IC del 95 %: [98,61 %; 99,42 %]) en personas que nunca vio durante el entrenamiento. Esto supera con holgura el objetivo del 85 %. Creemos que las claves del resultado son de diseño más que de capacidad: clasificar recortes de la mano en lugar del fotograma completo, construir la clase nula con manos de las mismas imágenes y medir el rendimiento con una partición por persona.

El estudio de ablación, repetido con dos semillas, indica que la normalización por lotes es el componente imprescindible: sin ella el entrenamiento es mucho más lento y, con el mismo número de épocas, comete más del doble de errores ($p < 0{,}001$). El *dropout* y el aumento de datos modifican la dinámica del entrenamiento como se espera de un regularizador, pero su efecto sobre la exactitud final es del mismo orden que la variación entre ejecuciones idénticas. La búsqueda manual de hiperparámetros llega a una conclusión parecida: de 9 cambios de tasa de aprendizaje, lote, capacidad y activación, solo el lote de 32 mejoró la validación, y esa mejora no se tradujo en el test. Con la mitad de filtros la red rinde igual con cuatro veces menos parámetros.

Frente a un ResNet18 preentrenado, con 4,7 veces más parámetros, la diferencia en test no es estadísticamente significativa ($p = 0{,}557$). Ambos modelos clasifican un recorte en pocos milisegundos incluso en CPU, de modo que cualquiera de los dos sirve para una aplicación en tiempo real. Grad-CAM confirma que la red basa sus decisiones en la forma de la mano, y los errores restantes se deben sobre todo a gestos realmente parecidos, a imágenes de baja calidad y a ruido de etiqueta.

**Trabajo futuro.**

- **Una red más pequeña.** Repetir con varias semillas las configuraciones más prometedoras de la búsqueda, en particular la de la mitad de filtros, que reduciría el tamaño del modelo

a la cuarta parte.

- **Medir el cambio de dominio.** Evaluar el sistema completo con las cajas de MediaPipe y con vídeos propios grabados con distintas iluminaciones, y comparar el resultado con el de test. Es el experimento que más falta para valorar el uso real.
- **Entrenar con cajas de detector.** Si el cambio de dominio es grande, entrenar con recortes generados por MediaPipe, o perturbar más las cajas durante el aumento de datos.
- **Más resolución.** Usar la versión de HaGRID a 512 px o superior para conservar detalle en manos pequeñas, que es lo que distingue *like* de *call*.
- **Clase nula más rica.** Revisar las etiquetas ruidosas de *no_gesture* y añadir posturas de transición entre gestos, que son las que más riesgo tienen de activar una orden por error.
- **Robot real.** Pasar de *turtlesim* a Gazebo o a un robot físico, que usa la misma interfaz de velocidades.

## Reparto del trabajo

El trabajo se repartió por ficheros con contratos fijos entre ambos (`src/common.py`), de modo que cada autor pudiera avanzar con datos o modelos simulados sin esperar al otro.

- **Daniel Rodríguez Alonso**: descarga y verificación de las anotaciones (`download.py`, `show_boxes.py`), recorte de manos (`prepare_data.py`), equilibrado y partición por persona (`make_splits.py`), carga de datos y aumento (`dataset.py`), evaluación y curvas (`evaluate.py`, `plot_curves.py`), Grad-CAM (`gradcam.py`), análisis estadístico y de coste (`analysis.py`) y ejecución de los entrenamientos de la ablación y de la referencia.
- **Carlos Falcón Brito**: propuesta inicial del proyecto, arquitectura GestureCNN y ResNet18 (`model.py`), datos simulados para desarrollar sin el conjunto real (`mock_data.py`), bucle de entrenamiento con parada anticipada (`train.py`), configuraciones de la ablación y lanzador de experimentos (`configs/`, `run_experiments.py`) y demo con ROS 2 (apartado 6).

**Uso de herramientas de IA.** Se utilizó un asistente de IA (Claude, de Anthropic) como apoyo en la programación, en la ejecución de los experimentos y en la redacción de este informe. Todas las cifras proceden de los ficheros de resultados del repositorio, generados por los *scripts* descritos. La responsabilidad sobre el contenido corresponde a los autores.

**Código.** El código, las configuraciones, la partición y los resultados están en `https://github.com/danirodriguezz/gestos-cnn-robot`. Las cifras de este documento se generan automáticamente a partir de `results/` con `src/analysis.py`.

## Referencias

[1] Pramod Kumar Pisharady and Martin Saerbeck. Recent methods and databases in vision-based hand gesture recognition: A review. *Computer Vision and Image Understanding*, 141: 152–165, 2015.

[2] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), 2012.

[3] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[4] Alexander Kapitanov, Karina Kvanchiani, Alexander Nagaev, Roman Kraynov, and Andrei Makhliarchuk. HaGRID – HAnd gesture recognition image dataset. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2024.

[5] Christian J. Mills. hagrid-sample-30k-384p: subconjunto de HaGRID a 384 px. Hugging Face Datasets, `https://huggingface.co/datasets/cj-mills/hagrid-sample-30k-384p`. Consultado en octubre de 2026.

[6] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. MediaPipe hands: On-device real-time hand tracking. *arXiv preprint arXiv:2006.10214*, 2020.

[7] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*, 2015.

[8] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *PMLR*, pages 448–456, 2015.

[9] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.

[10] Jonathan Tompson, Ross Goroshin, Arjun Jain, Yann LeCun, and Christoph Bregler. Efficient object localization using convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 648–656, 2015.

[11] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. In *International Conference on Learning Representations (ICLR)*, 2014.

[12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[13] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009.

[14] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, 2014.

[15] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, 2017.

[16] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[17] Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998.

[18] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[19] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6:60, 2019.

[20] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[21] Lutz Prechelt. Early stopping – but when? In *Neural Networks: Tricks of the Trade*, volume 1524 of *Lecture Notes in Computer Science*, pages 55–69. Springer, 1998.

[22] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (ELUs). In *International Conference on Learning Representations (ICLR)*, 2016.

[23] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), 2022.